

Pistachio Supply Chain Optimization

Raissa Gonçalves Diniz¹ and André Costa Batista^{1,2}

¹Operations Research and Complex Systems Laboratory, Universidade Federal de Minas Gerais, Brazil.

²Department of Electrical Engineering, Universidade Federal de Minas Gerais, Av. Antônio Carlos 6627, Belo Horizonte, 31270-010, Minas Gerais, Brazil.

Contributing authors: raissagd@ufmg.br; andre-costa@ufmg.br;

Abstract

The abstract serves both as a general introduction to the topic and as a brief, non-technical summary of the main results and their implications. Authors are advised to check the author instructions for the journal they are submitting to for word limits and if structural elements like subheadings, citations, or equations are permitted.

Keywords: keyword1, Keyword2, Keyword3, Keyword4

1 Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed cursus ullamcorper ante vel tempor. Curabitur tincidunt leo ligula, at faucibus sem porttitor non. Proin sodales, ex malesuada dictum cursus, libero massa feugiat erat, convallis sollicitudin ante arcu non ante. Curabitur vitae neque pretium, placerat quam quis, porttitor metus. In suscipit orci at lectus sollicitudin, vel porta sem tempor. Integer posuere, libero non eleifend imperdiet, nisl elit ultrices dolor, auctor fringilla libero justo eget leo. Curabitur maximus libero pharetra felis porta, quis elementum massa fermentum.

2 Problem definition

The proposed network includes producers, processing centers, pistachio factories, oil extraction centers, composting centers, and customers. Pistachios are transported from

orchards to processing centers, producing open-mouth pistachios, raw kernels, and waste, which are then sent to factories, oil extraction centers, and composting centers, respectively. Open-mouth pistachios are roasted, packed, and shipped to customers, while pistachio oil is delivered to oil customers and cosmetic factories. Waste is converted into compost for distribution. The model aims to minimize total costs (1), including opening, transportation, and production costs, under the following capacity and demand constraints with no allowed shortages:

$$\min z_1 + z_2 + z_3 \quad (1)$$

$$\text{s.t.} \quad \sum_{j=1}^J X_{ij} \leq Cpa_i, \quad \forall i \in I \quad (2)$$

$$\sum_{i=1}^I X_{ij} \leq Cpu_j \times U_j, \quad \forall j \in J \quad (3)$$

$$\sum_{e=1}^E Ow_{eq} + \sum_{j=1}^J Gw_{jq} \leq Cpy_q \times T_q, \quad \forall q \in Q \quad (4)$$

$$\sum_{j=1}^J Go_{jk} \leq Cpw_k \times W_k, \quad \forall k \in K \quad (5)$$

$$\sum_{j=1}^J Gr_{je} \leq Cpr_e \times R_e, \quad \forall e \in E \quad (6)$$

$$\sum_{e=1}^E Oen_2 \leq Cpv_s \times V_s, \quad \forall s \in S \quad (7)$$

$$\sum_{k=1}^K Go_{jk} + \sum_{e=1}^E Gr_{je} + \sum_{q=1}^Q Gw_{jq} \leq \sum_{i=1}^I X_{ij}, \quad \forall j \in J \quad (8)$$

$$\sum_{k=1}^K Go_{jk} = (1 - \beta) \sum_{i=1}^I X_{ij} \theta_1, \quad \forall j \in J \quad (9)$$

$$\sum_{e=1}^E Gr_{je} = (1 - \beta) \sum_{i=1}^I X_{ij} \theta_2, \quad \forall j \in J \quad (10)$$

$$\sum_{q=1}^Q Gw_{jq} = \sum_{i=1}^I X_{ij} \theta_3, \quad \forall j \in J \quad (11)$$

$$\theta_1 + \theta_2 + \theta_3 = 1 \quad (12)$$

$$\sum_{n_1=1}^{N_1} P_{kn_1} \leq \sum_{j=1}^J Go_{jk} \times \gamma k, \quad \forall k \in K \quad (13)$$

$$\sum_{n_2=1}^{N_2} O_{en_2} + \sum_{s=1}^S O_{ces} \leq \sum_{j=1}^J Gr_{je} \times (1 - \lambda), \quad \forall e \in E \quad (14)$$

$$\sum_{q=1}^Q Ow_{eq} \leq \sum_{j=1}^J Gr_{je} \times \lambda k, \quad \forall e \in E \quad (15)$$

$$\sum_{n_3=1}^{N_3} L_{sn_1} \leq \sum_{e=1}^E O_{ces} \times \gamma s, \quad \forall s \in S \quad (16)$$

$$\sum_{m=1}^M D_{qm} \leq \left(\sum_{j=1}^J Gw_{jq} + \sum_{e=1}^E Ow_{eq} \right) \times \gamma q, \quad \forall q \in Q \quad (17)$$

$$\sum_{k=1}^K P_{kn_1} \leq Dp_{n_1}, \quad \forall n_1 \in N_1 \quad (18)$$

$$\sum_{e=1}^E O_{en_2} \leq Du_{n_2}, \quad \forall n_2 \in N_2 \quad (19)$$

$$\sum_{s=1}^S L_{sn_3} \leq Ds_{n_3}, \quad \forall n_3 \in N_3 \quad (20)$$

$$\sum_{q=1}^Q D_{qm} \leq Dc_m, \quad \forall m \in M \quad (21)$$

- The objective function (1) minimizes the total costs of the supply chain, including opening, transportation, and production costs. The costs are divided into three components: z_1 (22), z_2 (23), and z_3 (24), which represent the costs of opening facilities, transportation, and production, respectively as shown in the following equations:

$$z_1 = \sum_{j=1}^J Fu_j \times U_j + \sum_{q=1}^Q Fy_q \times Y_q + \sum_{k=1}^K Fw_k \times W_k + \sum_{e=1}^E Fr_e \times R_e + \sum_{s=1}^S Fv_s \times V_s \quad (22)$$

$$\begin{aligned} z_2 = & \sum_{i=1}^I \sum_{j=1}^J CI_i \times X_{ij} + \sum_{j=1}^J \sum_{k=1}^K Cu_{1j} \times Go_{jk} + \sum_{j=1}^J \sum_{e=1}^E Cu_{2j} \times Gr_{je} \\ & + \sum_{q=1}^Q \sum_{m=1}^M Cy_q \times D_{qm} + \sum_{k=1}^K \sum_{n_1=1}^{N_1} Cw_k \times P_{kn_1} \\ & + \sum_{e=1}^E Cr_e \left(\sum_{n_2=1}^{N_2} O_{en_2} + \sum_{s=1}^S O_{ces} \right) + \sum_{s=1}^S \sum_{n_3=1}^{N_3} Cv_s \times L_{sn_3} \end{aligned} \quad (23)$$

$$\begin{aligned}
z_3 = & \sum_{i=1}^I \sum_{j=1}^J CX_{ij} \times X_{ij} + \sum_{j=1}^J \sum_{k=1}^K CK_{jk} \times Go_{jk} + \sum_{j=1}^J \sum_{q=1}^Q CJ_{jq} \times Gw_{jq} \\
& + \sum_{e=1}^E \sum_{s=1}^S CS_{es} \times Oc_{es} + \sum_{e=1}^E \sum_{n_2=1}^{N_2} CN_{en_2} \times Oc_{en_2} + \sum_{e=1}^E \sum_{q=1}^Q CQ_{eq} \times Ow_{eq} \\
& + \sum_{s=1}^S \sum_{n_3=1}^{N_3} Cl_{sn_3} \times L_{sn_3} + \sum_{k=1}^K \sum_{n_1=1}^{N_1} Cp_{kn_1} \times P_{kn_1} + \sum_{q=1}^Q \sum_{m=1}^M Cd_{qm} \times D_{qm} \quad (24)
\end{aligned}$$

- The variables R_j , Y_q , W_k , Z_e , and V_s are binary decision variables that indicate whether certain elements in the model are active. For example, $R_j = 1$ if facility j is open, and 0 otherwise. The other variables in the model are continuous and positive, representing real numbers greater than or equal to zero, used for modeling flows, quantities, and other measurable factors.
- The constraints (2)-(7) address the finite capacities and demands of each center within the supply chain. Specifically, constraint (2) limits the quantity of products shipped from producers to distribution centers. Constraints (3)-(7) then ensure that the quantities of products delivered to processing centers, compost centers, pistachio factories, oil extraction centers, and cosmetic factories do not exceed the capacities of these centers.
- Products sent from any center must not exceed the orders received by that center. Therefore, constraint (8) stipulates that the quantity of products shipped from a processing center must be less than or equal to the quantity delivered to other centers. Constraints (9)-(12) specify the quantities of open-mouth pistachios, raw kernels, and waste produced through the dehulling process at each processing center. Similarly, constraints (13)-(17) regulate the flows within pistachio factories, oil extraction centers, cosmetic factories, and composting centers.
- Finally, to ensure that customer demand is met, constraints (18)-(21) guarantee that the products delivered to pistachio customers, pistachio oil customers, cosmetic customers, and compost markets meet or exceed their respective demands.

3 Heuristic approach

In this section, we present the methodology employed to address the optimization problem. The proposed approach integrates priority-based encoding and the Variable Neighborhood Search (VNS) algorithm, which together form the core of our solution strategy. We begin by detailing the solution representation, which utilizes priority-based encoding to efficiently manage and rank decision variables, allowing for effective exploration of the solution space. This encoding scheme is crucial in guiding the search process and maintaining feasibility while optimizing the objective function. Following this, we delve into the VNS algorithm, a robust metaheuristic known for its flexibility and capacity to escape local optima through systematic changes in neighborhood structures. The algorithm dynamically adjusts between multiple neighborhood structures, each designed to explore different aspects of the solution landscape, thereby enhancing the search process. We will also discuss the specific configurations and parameters

used, as well as the design of neighborhood operators tailored to the problem characteristics. This comprehensive approach aims to deliver a high-quality solution by leveraging the strengths of priority-based encoding and the adaptability of VNS.

3.1 Solution representation

In this work, a priority-based encoding scheme is used to represent the solution space effectively, drawing on chromosome-based representations where each gene corresponds to a node (source or depot) in the transportation problem [1]. The transportation problem is modeled as a tree structure, with nodes representing sources or depots and edges representing transportation arcs. The primary objective is to construct a transportation tree that meets all depot demands while minimizing total transportation costs.

Each chromosome encodes the priority of nodes, guiding the construction of the transportation tree by establishing the most cost-effective connections first. The decoding process starts by identifying the node with the highest priority in the chromosome and connecting it to the corresponding node with the lowest transportation cost. For instance, if Depot 1 has the highest priority, it is linked to the source that offers the lowest cost connection to Depot 1. The shipment amount is determined by the minimum of the source’s capacity and the depot’s demand. Once the shipment is fulfilled, the depot’s priority is updated (or set to zero if its demand is satisfied), and the process continues with the next highest priority node. This sequential process of arc appending and priority updating repeats until all depots’ demands are satisfied, resulting in a complete transportation tree.

The supply chain is determined using a step-by-step, backward approach, starting from consumer demand and moving upward toward producers. This method ensures resource allocation is optimized at each stage, preventing excess or unnecessary material flow. For example, after determining the quantity of open-mouth pistachios required by consumers, we work backward to calculate the factories’ processing needs, ensuring their capacities are not overloaded. This then informs how much material the processing centers must supply, aligning each upstream stage with the final demand and optimizing resource use throughout the chain.

As shown in Figure 1, the decoding process begins with the flow of pistachio products from factories to consumers, ensuring that all constraints are met: consumer demand is satisfied, the total amount of product leaving the factory does not exceed the pistachio received from processing centers multiplied by the factory’s production rate, and the pistachio sent to any active factory is within its capacity. Likewise, the distribution of cosmetic products from factories to consumers and the transfer of oil from extraction centers to both oil consumers and cosmetic factories adhere to the same principles, ensuring all constraints are satisfied.

The flow from processing centers to pistachio factories and oil extraction centers is regulated by constraints ensuring that the total pistachios leaving a processing center do not exceed the harvested amount sent by producers. Harvested pistachios are divided into three parts, each assigned to a product type, with a $(1 - \beta)$ loss factor applied to pistachios with shells and pits. Factories can receive more pistachios than they process, but must meet a minimum intake, while oil extraction centers send

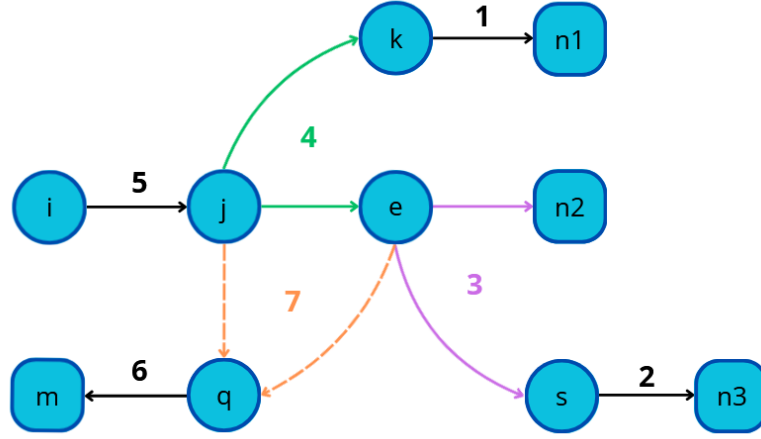


Fig. 1: Decodification steps of the transportation tree

no more than $(1 - \lambda)$ of the kernels to consumers and cosmetics factories, with the remaining λ going to composting. All processing centers must respect their capacity limits to ensure efficient distribution.

The decoding process for this specific flow is more complex and requires a targeted approach. First, we must determine the flow of in-shell pistachios from processing centers to pistachio factories, and the flow of kernels from processing centers to oil extraction centers. Using the given constraints, we calculate the quantity of harvested pistachios each processing center must receive to meet the demands of both pistachio factories and oil extraction centers.

If the factory demand exceeds that of the oil extraction centers, the processing center must prioritize meeting the exact demand of the factories. Since harvested pistachios are divided into three proportional parts, basing the allocation solely on the oil extraction center's demand would leave the factories undersupplied. The priority should be given to the entity with the greatest need, adjusting the remaining flow accordingly, as constraints allow for over-supply but not shortages.

Thus, we need to adjust the quantity of kernels shipped from the processing centers to the oil extraction centers, ensuring enough is sent to satisfy the restrictions. This involves calculating the total amount of kernels required, identifying the oil extraction center with the lowest transportation cost, and directing the additional flow there to minimize costs.

If, instead, the oil extraction centers have higher demand, the process is reversed: the pistachio factories' flow is adjusted accordingly to balance the supply while minimizing transportation costs.

Next, the flow of harvested pistachios from producers to processing centers is also governed by constraints ensuring that no producer exceeds its production capacity and no processing center receives more than it can process. While the capacity of each producer limits the supply, the demand for each processing center has already been

determined in a previous step. If producers' capacities are insufficient to meet the total demand, the solution becomes infeasible. However, due to the constraints on product allocation between processing centers, this flow will always satisfy the condition of equality.

Similarly, the flow of fertilizer from composting centers to consumers is governed by constraints that ensure composting centers do not receive more waste than they can handle. The amount of fertilizer sent must be less than or equal to the waste received from processing and oil extraction centers, adjusted by the composting center's production rate. Finally, for the waste flow from processing and oil extraction centers to composting centers, constraints also ensure that no composting center receives more waste than its capacity. Waste from each processing center is assigned to the composting center with the lowest associated cost, ensuring that no composting center receives more waste than required. If the waste sent to a composting center falls short of its needs, the demand is adjusted accordingly. Conversely, if excess waste is sent, the demand is set to zero. Once all processing centers have allocated their waste, any remaining composting needs are fulfilled by oil extraction centers, considering their available capacity. If no additional waste is needed, no waste is sent from the oil extraction centers. Finally, the total cost of transporting waste is updated based on the assignments, ensuring efficient waste management across the supply chain.

3.2 VNS algorithm

The Variable Neighborhood Search (VNS) algorithm is a powerful metaheuristic that combines local search with systematic changes in neighborhood structures to explore the solution space effectively. By iteratively applying different neighborhood operators, VNS can escape local optima and discover high-quality solutions through continuous diversification and intensification phases.

The algorithm begins by initializing a solution and setting the maximum neighborhood size and time limit. It then enters a loop, where it repeatedly applies the shaking and local search procedures to generate new candidate solutions. If a better solution is found, it replaces the current best solution and resets the neighborhood size. Otherwise, the algorithm increases the neighborhood size and continues the search. The process repeats until the time limit is reached, at which point the best found solution is returned. Algorithm 1 provides a detailed outline of the VNS algorithm.

3.3 Neighborhood structures

In the context of the Variable Neighborhood Search (VNS) algorithm, neighborhood operators play a crucial role in exploring the solution space by systematically altering the current solution to generate new candidate solutions. The effectiveness of VNS relies heavily on the design and application of these operators, as they determine the diversity and direction of the search process. In this study, we utilize four classic neighborhood operators: Swap, Reversion, Insertion, and Slide. Each operator perturbs the solution in a distinct manner, allowing the algorithm to escape local optima and explore different regions of the search space. By iteratively applying these operators,

Algorithm 1 Variable Neighborhood Search (VNS)

Require: Initial solution x , maximum neighborhood size $k_{\max}$, time limit $t_{\max}$

Ensure: Best found solution x^*

```
1:  $x^* \leftarrow x$ 
2: repeat
3:    $k \leftarrow 1$ 
4:   repeat
5:      $x' \leftarrow \text{Shake}(x, k)$  ▷ Shaking
6:      $x'' \leftarrow \text{LocalSearch}(x')$  ▷ Local search
7:     if  $f(x'') < f(x^*)$  then
8:        $x^* \leftarrow x''$ 
9:        $k \leftarrow 1$  ▷ Reset neighborhood size
10:    else
11:       $k \leftarrow k + 1$ 
12:    end if
13:  until  $k > k_{\max}$ 
14:   $t \leftarrow \text{CpuTime}()$ 
15: until  $t > t_{\max}$ 
```

VNS enhances its capability to find high-quality solutions through continuous diversification and intensification phases. The neighborhood structures used are detailed below, and illustrated in Figure 2.:

- **Swap:** The swap operator exchanges the positions of two selected elements within the chromosome. In the example, the selected elements are 6 and 3. This operator is useful for making quick adjustments to the order of elements without altering the overall structure significantly.
- **Reversion:** The reversion operator reverses the order of all elements between the two selected indices. In this case, with elements 6 and 3 chosen, the operator reverses the sequence between these two points. This operation can effectively rearrange subsequences, potentially moving the solution closer to a local optimum by drastically altering a segment of the sequence.
- **Insertion:** In this operation, the unit in the second selected position is relocated immediately after the unit in the first position. Subsequently, all units that were originally between these positions are shifted accordingly to the right. This allows the algorithm to make focused adjustments by repositioning a specific element within the sequence while maintaining the overall order of the other elements.
- **Slide:** This operator begins by selecting two units of a solution randomly, like the other three. The first unit is removed, and the units that were initially located between the two selected units are shifted toward the start position of the first unit. Finally, the initially removed unit is placed at the position of the second unit. This sliding mechanism creates a subtle yet impactful reordering of the solution elements, which can enhance the search process by generating new configurations with minimal disruption.

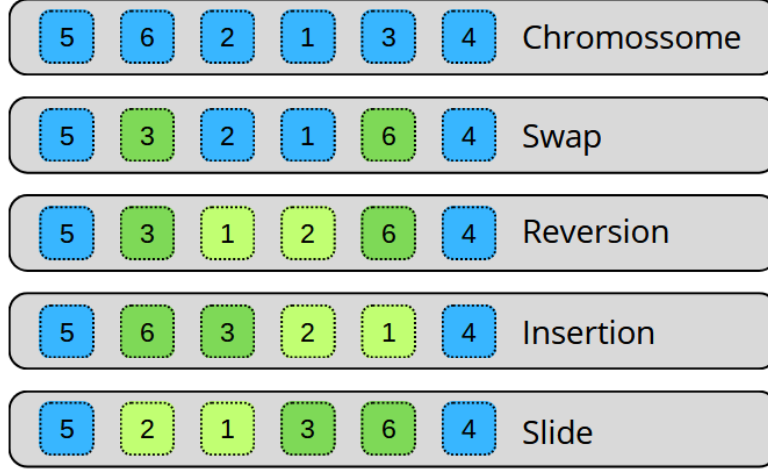


Fig. 2: Neighborhood structures used in the VNS Algorithm

4 Results

Our results demonstrate that the Variable Neighborhood Search (VNS) algorithm outperforms both the Genetic Algorithm (GA) and Gurobi when considering execution time and memory efficiency, especially on larger instances. While Gurobi guarantees optimal solutions, its memory consumption increases significantly with larger problems, often making it impractical. VNS provides high-quality solutions with a substantial reduction in both execution time and memory usage, making it a more viable choice for large-scale problems. Although GA remains popular in literature, its performance falters as problem sizes increase, both in terms of time and solution quality.

4.1 Comparison with Gurobi

As shown in Table 1, Gurobi efficiently finds optimal solutions for smaller instances, such as 100 and 200. The solution difference between VNS and Gurobi is rather little for these smaller cases — 4.19% for 100 instances and 8.22% for 200 instances. However, as the problem size increases, the difference in solution quality becomes larger, with VNS being 20.12% away from the optimal solution at 800 instances.

When examining execution time, the results clearly highlight Gurobi’s inefficiency on larger instances. For example, at 400 instances, Gurobi takes over 70,000 seconds to solve the problem, while VNS takes just over 2,000 seconds, offering a remarkable 97.13% reduction in time. By the time we reach 1600 instances, Gurobi is unable to find a solution due to its excessive resource demands, while VNS continues to provide a feasible solution in approximately 17,766 seconds.

These results emphasize that although Gurobi produces the best solutions for small instances, it becomes impractical for larger problems due to its memory usage and

excessive execution times. VNS, while not optimal, offers a more practical and scalable solution for larger instances.

4.2 Comparison between VNS and GA

The performance of the Genetic Algorithm (GA) can be seen in Table 2. For small instances, such as 100, the difference between VNS and GA is negligible—only 0.71%. However, as the problem size grows, GA’s performance gradually worsens compared to VNS. By 800 instances, the difference in solution quality rises to 5.26%, indicating that VNS provides consistently better solutions for larger problems.

It is important to highlight that GA takes considerably longer to run compared to VNS. The nature of the GA, which relies on iterative population-based mechanisms such as selection, crossover, and mutation, demands a high number of epochs to reach an acceptable solution. This results in prolonged execution times, particularly for larger problem sizes. In contrast, VNS, with its focused exploration of neighborhood structures, efficiently narrows down the solution space, allowing for faster convergence. This significant difference in runtime makes VNS a more practical choice for larger problem instances, as GA’s prolonged execution time limits its scalability and applicability in real-world scenarios where timely solutions are crucial.

In summary, while GA can compete with VNS in smaller problem instances, it lags behind as the complexity increases. VNS provides not only better solutions but also handles larger instances more efficiently, making it the more robust and scalable option for large-scale optimization problems.

Table 1: Average solution values and execution times of the VNS algorithm compared to the Exact Algorithm (Gurobi).

Instance	Solution value			Execution time		
	VNS	Gurobi	Difference	VNS	Gurobi	Difference
100	7.356017e+06	7059878.15	4.19%	240.576739s	261s	-7.83%
200	1.488441e+08	137535570.42	8.22%	714.499149s	3834s	-81.36%
400	3.075344e+08	270662192.84	13.62%	2015.728858s	70339s	-97.13%
800	6.573093e+08	547200537.38	20.12%	5537.291645s	11836s	-53.22%
1600	1.364413e+09	-	-	17766.873715s	-	-

5 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed cursus ullamcorper ante vel tempor. Curabitur tincidunt leo ligula, at faucibus sem porttitor non. Proin sodales, ex malesuada dictum cursus, libero massa feugiat erat, convallis sollicitudin ante arcu non ante. Curabitur vitae neque pretium, placerat quam quis, porttitor metus. In suscipit orci at lectus sollicitudin, vel porta sem tempor. Integer posuere, libero non eleifend imperdiet, nisl elit ultrices dolor, auctor fringilla libero justo eget leo. Curabitur maximus libero pharetra felis porta, quis elementum massa fermentum.

Table 2: Average solution values of the VNS algorithm compared to the Genetic Algorithm (GA).

Instance	Solution value		
	VNS	GA	Difference
100	74105277.59	74630611.96	0.71%
200	150165899.77	155311143.01	3.37%
400	307527336.72	324315714.06	5.31%
800	657000372.14	692477371.5	5.26%
1600	1363240244.41	-	-

6 Methods

Topical subheadings are allowed. Authors must ensure that their Methods section includes adequate experimental and characterization data necessary for others in the field to reproduce their work. Authors are encouraged to include RIIDs where appropriate.

Ethical approval declarations (only required where applicable) Any article reporting experiment/s carried out on (i) live vertebrate (or higher invertebrates), (ii) humans or (iii) human samples must include an unambiguous statement within the methods section that meets the following requirements:

1. Approval: a statement which confirms that all experimental protocols were approved by a named institutional and/or licensing committee. Please identify the approving body in the methods section
2. Accordance: a statement explicitly saying that the methods were carried out in accordance with the relevant guidelines and regulations
3. Informed consent (for experiments involving humans or human tissue samples): include a statement confirming that informed consent was obtained from all participants and/or their legal guardian/s

If your manuscript includes potentially identifying patient/participant information, or if it describes human transplantation research, or if it reports results of a clinical trial then additional information will be required. Please visit (<https://www.nature.com/nature-research/editorial-policies>) for Nature Portfolio journals, (<https://www.springer.com/gp/authors-editors/journal-author/journal-author-helpdesk/publishing-ethics/14214>) for Springer Nature journals, or (<https://www.biomedcentral.com/getpublished/editorial-policies#ethics+and+consent>) for BMC.

7 Discussion

Discussions should be brief and focused. In some disciplines use of Discussion or ‘Conclusion’ is interchangeable. It is not mandatory to use both. Some journals prefer a section ‘Results and Discussion’ followed by a section ‘Conclusion’. Please refer to Journal-level guidance for any specific requirements.

8 Conclusion

Conclusions may be used to restate your hypothesis or research question, restate your major findings, explain the relevance and the added value of your work, highlight any limitations of your study, describe future directions for research and recommendations.

In some disciplines use of Discussion or 'Conclusion' is interchangeable. It is not mandatory to use both. Please refer to Journal-level guidance for any specific requirements.

Supplementary information. If your article has accompanying supplementary file/s please state so here.

Authors reporting data from electrophoretic gels and blots should supply the full unprocessed scans for key as part of their Supplementary information. This may be requested by the editorial team/s if it is missing.

Please refer to Journal-level guidance for any specific requirements.

Acknowledgements. Acknowledgements are not compulsory. Where included they should be brief. Grant or contribution numbers may be acknowledged.

Please refer to Journal-level guidance for any specific requirements.

Declarations

Some journals require declarations to be submitted in a standardised format. Please check the Instructions for Authors of the journal to which you are submitting to see if you need to complete this section. If yes, your manuscript must contain the following sections under the heading 'Declarations':

- Funding
- Conflict of interest/Competing interests (check journal-specific guidelines for which heading to use)
- Ethics approval and consent to participate
- Consent for publication
- Data availability
- Materials availability
- Code availability
- Author contribution

If any of the sections are not relevant to your manuscript, please include the heading and write 'Not applicable' for that section.

Editorial Policies for:

Springer journals and proceedings: <https://www.springer.com/gp/editorial-policies>

Nature Portfolio journals: <https://www.nature.com/nature-research/editorial-policies>

Scientific Reports: <https://www.nature.com/srep/journal-policies/editorial-policies>

BMC journals: <https://www.biomedcentral.com/getpublished/editorial-policies>

Appendix A Section title of first appendix

An appendix contains supplementary information that is not an essential part of the text itself but which may be helpful in providing a more comprehensive understanding of the research problem or it is information that is too cumbersome to be included in the body of the paper.

References

- [1] Gen, M., Altıparmak, F. & Lin, L. A genetic algorithm for two-stage transportation problem using priority-based encoding. *OR spectrum* **28**, 337–354 (2006).